

Recálculo de Orçamento: Custos, Frete e Vendas

https://gemini.google.com/app/439bfadfad3c9bd8

```
User prompt: qua é o comprtamento dessa função // Recálculo dinâmico do orçamento por somatório. // Lê todos os itens do orçamento, soma os custos_nota,
recalcula o frete B2B (Kapazi) // sobre o total, rateia proporcionalmente, aplica o markup (efetivo quando newMargem // informado), desconto e frete B2C, e
atualiza itens + cabeçalho ORCA. function Orcamento_Recalcular_Totais { input { int orca_id? { table = "Orca" } } // Markup EFETIVO desejado (%).
Se omitido, mantém a margem de cada item decimal newMargem? } stack { db.get Orca { field_name = "id" field_value = $input.orca_id } as
$Orca_0 db.query item { where = $db.item.orca_id == $input.orca_id sort = {item.id: "asc"} return = {type: "list" output = { "id"
"created_at" "orca_id" "produto_id" "ipi" "imp" "vlr_custo" "und_produto" "larg" "comp" "larg_fc" "comp_fc"
"borda_id" "vlr_cst_borda" "und_borda" "tipo_fator_id" "fator_de_corte_id" "detalhe_id" "variacao_id" "margem" "qtd"
"vlr_cst_unit" "vlr_cst_unit_ipi" "vlr_cst_unit_imp" "vlr_vnd_unit" "vlr_vnd_unit_ipi" "vlr_vnd_unit_imp" "vlr_lucro_unit"
"vlr_vnd_unit_b2b" "descricao" "area_user" "area_calc" "vlr_cst_nota_unit" "vlr_cst_entrada_unit" "valor_difal_unit"
"vlr_credito_icms_unit" "aliq_inter" "aliq_interna" "perc_difal" "vlr_frete_b2b_unit" "vlr_st_unit" "vlr_custo_fiscal_unit"
"eh_importado" "perc_margem_real" } } as $itens // 1. Preparação da soma total dos itens para validação da faixa de Frete B2B api.lambda {
code = "" const itens = $var.itens || []; const orca = $var.Orca_0 || {}; const desconto = Number(orca.desconto) || 0; const frtB2C =
Number(orca.frtB2C) || 0; const inputMargem = $input ? Number($input.newMargem) : 0; const markupAlvo = (inputMargem > 0) ? inputMargem :
(Number(orca.margem) || 0); const itensComCustoBase = itens.map(item => { const qtd = Number(item.qtd) || 1; const vlr_cst_nota_unit =
Number(item.vlr_cst_nota_unit) || Number(item.vlr_cst_nota_ipi) || Number(item.vlr_cst_nota_imp) || 0; const custo_total_nota_item = vlr_cst_nota_unit * qtd;
return { ...item, qtd, vlr_cst_nota_unit, custo_total_nota_item }); }); const total_custo_nota =
itensComCustoBase.reduce((acc, i) => acc + i.custo_total_nota_item, 0); return { total_custo_nota: Number(total_custo_nota.toFixed(4)),
itensComCustoBase, markupAlvo, desconto, frtB2C }; // 2. Cálculo dinâmico do Frete
B2B Kapazi function.run fCalculaFrete { input = { valor_total_compra: $soma_itens.total_custo_nota seu_frete_minimo : 52 } } as
$frete_b2b_total // 3. Rateio do Frete, Cálculo dos Impostos (ST/DIFAL/Crédito) e Venda com Markup api.lambda { code = "" const
itensComCustoBase = $var.soma_itens.itensComCustoBase || []; const total_custo_nota = $var.soma_itens.total_custo_nota || 0; const frete_b2b_total =
Number($var.frete_b2b_total) || 0; const orca = $var.Orca_0 || {}; const desconto = Number(orca.desconto) || 0; const frtB2C =
Number(orca.frtB2C) || 0; const inputMargem = $input ? Number($input.newMargem) : 0; const markupAlvo = (inputMargem > 0) ? inputMargem :
(Number(orca.margem) || 0); // Rateio do Frete B2B e Custos Fiscais / Entrada const itensComCustoEntrada =
itensComCustoBase.map(item => { const qtd = Number(item.qtd) || 1; const difal_unit = Number(item.valor_difal_unit) || 0; const credito_unit =
Number(item.vlr_credito_icms_unit) || 0; const st_unit = Number(item.vlr_st_unit) || Number(item.vlr_st) || 0; const cst_nota_unit =
Number(item.vlr_cst_nota_unit) || 0; const custo_total_nota_item = cst_nota_unit * qtd; const ratio = total_custo_nota > 0 ?
(custo_total_nota_item / total_custo_nota) : 0; const frete_rateado_total = frete_b2b_total * ratio; const vlr_frete_b2b_unit = frete_rateado_total / qtd;
// Custo Fiscal Unitário e Total (Produto + DIFAL - Crédito + ST) const vlr_custo_fiscal_unit = cst_nota_unit + difal_unit - credito_unit +
st_unit; const custo_fiscal_total = vlr_custo_fiscal_unit * qtd; // Custo Real de Entrada Unitário e Total (+ Frete B2B) const
vlr_cst_entrada_unit = vlr_custo_fiscal_unit + vlr_frete_b2b_unit; const custo_entrada_total = vlr_cst_entrada_unit * qtd; return { ...item,
qtd, cst_nota_unit, custo_total_nota_item, difal_unit, credito_unit, st_unit, vlr_frete_b2b_unit,
frete_rateado_total, vlr_custo_fiscal_unit, custo_fiscal_total, vlr_cst_entrada_unit, custo_entrada_total }); });
const cst_tot = itensComCustoEntrada.reduce((acc, i) => acc + i.custo_entrada_total, 0); // Acumulados do Cabeçalho const vlr_st_tot =
itensComCustoEntrada.reduce((acc, i) => acc + (i.st_unit * i.qtd), 0); const valor_difal_tot = itensComCustoEntrada.reduce((acc, i) => acc + (i.difal_unit *
i.qtd), 0); const vlr_credito_icms_tot = itensComCustoEntrada.reduce((acc, i) => acc + (i.credito_unit * i.qtd), 0); const vlr_custo_fiscal_tot =
itensComCustoEntrada.reduce((acc, i) => acc + i.custo_fiscal_total, 0); // Venda Bruta e Rateio do Desconto const itensComVendaBruta =
itensComCustoEntrada.map(item => { const venda_bruta_item = item.custo_entrada_total * (1 + (markupAlvo / 100)); return { ...item,
venda_bruta_item }; }); const total_venda_bruta = itensComVendaBruta.reduce((acc, i) => acc + i.venda_bruta_item, 0); const
itensComVendaFinal = itensComVendaBruta.map(item => { const ratioVenda = total_venda_bruta > 0 ? (item.venda_bruta_item / total_venda_bruta) : 0;
const desconto_rateado_item = desconto * ratioVenda; const venda_liquida_item = item.venda_bruta_item - desconto_rateado_item; const
vlr_vnd_unit = venda_liquida_item / item.qtd; const vlr_lucro_unit = vlr_vnd_unit - item.vlr_cst_entrada_unit; const perc_margem_real = vlr_vnd_unit
> 0 ? (vlr_lucro_unit / vlr_vnd_unit) * 100 : 0; return { ...item, venda_liquida_item, vlr_vnd_unit, vlr_lucro_unit,
perc_margem_real }; }); const vnd_tot = total_venda_bruta - desconto; const luc_tot = vnd_tot - cst_tot; const markup_efetivo =
cst_tot > 0 ? ((vnd_tot - cst_tot) / cst_tot) * 100 : markupAlvo; const margem_real_tot = vnd_tot > 0 ? (luc_tot / vnd_tot) * 100 : 0; return {
itensParaUpdate: ItensComVendaFinal.map(i => { id : i.id, vlr_cst_nota_unit : Number(i.cst_nota_unit.toFixed(4)),
vlr_st_unit : Number(i.st_unit.toFixed(4)), vlr_custo_fiscal_unit : Number(i.vlr_custo_fiscal_unit.toFixed(4)), vlr_frete_b2b_unit :
Number(i.vlr_frete_b2b_unit.toFixed(4)), vlr_cst_entrada_unit : Number(i.vlr_cst_entrada_unit.toFixed(4)), vlr_vnd_unit :
Number(i.vlr_vnd_unit.toFixed(4)), vlr_vnd_unit_b2b : Number(i.vlr_vnd_unit_b2b.toFixed(4)), vlr_lucro_unit :
Number(i.vlr_lucro_unit.toFixed(4)), margem : Number(markup_efetivo.toFixed(4)), perc_margem_real :
Number(i.perc_margem_real.toFixed(4)) }, totais: { cst_tot : Number(cst_tot.toFixed(4)), vlr_st_tot :
Number(vlr_st_tot.toFixed(4)), valor_difal_tot : Number(valor_difal_tot.toFixed(4)), vlr_credito_icms_tot :
Number(vlr_credito_icms_tot.toFixed(4)), vlr_custo_fiscal_tot : Number(vlr_custo_fiscal_tot.toFixed(4)), vnd_tot :
Number(vnd_tot.toFixed(4)), luc_tot : Number(luc_tot.toFixed(4)), vnd_B2B_tot : Number(vnd_tot.toFixed(4)),
vnd_B2B_B2C_tot : Number(vnd_tot + frtB2C).toFixed(4)), margem : Number(markup_efetivo.toFixed(4)), margem_real_total :
Number(margem_real_tot.toFixed(4)), frete_b2b_total : Number(frete_b2b_total.toFixed(4)), desconto : Number(desconto.toFixed(4)),
frtB2C : Number(frtB2C.toFixed(4)), total_itens : itensComVendaFinal.length } }; // 4. Atualização dos itens com os novos valores unitários
foreach ($recalc.itensParaUpdate) { each as $supd { db.edit item { field_name =
"id" field_value = $supd.id enforce_hidden_fields = false data = { margem : $supd.margem vlr_vnd_unit :
$supd.vlr_vnd_unit vlr_lucro_unit : $supd.vlr_lucro_unit vlr_vnd_unit_b2b : $supd.vlr_vnd_unit_b2b vlr_cst_nota_unit :
$supd.vlr_cst_nota_unit vlr_cst_entrada_unit : $supd.vlr_cst_entrada_unit vlr_frete_b2b_unit : $supd.vlr_frete_b2b_unit vlr_st_unit :
$supd.vlr_st_unit vlr_custo_fiscal_unit : $supd.vlr_custo_fiscal_unit perc_margem_real : $supd.perc_margem_real } } as $item_editado
} } // 5. Atualização completa do cabeçalho ORCA (Custos + Vendas + Impostos Totais) db.add_or_edit Orca { field_name = "id" field_value =
$input.orca_id enforce_hidden_fields = false data = { margem : $recalc.totais.margem cst_tot : $recalc.totais.cst_tot
vnd_tot : $recalc.totais.vnd_tot luc_tot : $recalc.totais.luc_tot vnd_B2B_tot : $recalc.totais.vnd_B2B_tot vnd_B2B_B2C_tot :
$recalc.totais.vnd_B2B_B2C_tot vlr_st_tot : $recalc.totais.vlr_st_tot valor_difal_tot : $recalc.totais.valor_difal_tot vlr_credito_icms_tot :
$recalc.totais.vlr_credito_icms_tot vlr_custo_fiscal_tot : $recalc.totais.vlr_custo_fiscal_tot total_itens : $recalc.totais.total_itens } } as
$Orca_1 // 6. Consulta final para retorno dos itens ao front-end db.query item { join = { Produto : { table: "Produto" where:
$db.item.produto_id == $db.Produto.id } Material : { table: "Material" where: $db.Material.id == $db.Produto.material_id } Linha :
{ table: "Linha" type : "left" where: $db.Linha.id ==? $db.Produto.linha_id } Tipo : { table: "Tipo" type : "left"
where: $db.Tipo.id ==? $db.Produto.tipo_id } Nivel : { table: "Nivel" type : "left" where: $db.Nivel.id ==? $db.Produto.nivel_id }
Borda : { table: "Borda" type : "left" where: $db.Borda.id ==? $db.item.borda_id } } where = $db.item.orca_id ==
$input.orca_id sort = {item.id: "asc"} eval = { Descricao: $db.Material.nome|concat:"|concat:$db.Linha.nome|concat:"|concat:$db.Tipo.nome|concat:"|concat:$db.Nivel.nome|concat:"|concat:$db.Borda.nome } return = {type: "list" output = { "id"
"created_at" "orca_id" "produto_id" "ipi" "imp" "vlr_custo" "und_produto" "larg" "comp" "larg_fc" "comp_fc"
"borda_id" "vlr_cst_borda" "und_borda" "tipo_fator_id" "fator_de_corte_id" "detalhe_id" "variacao_id" "margem" "qtd"
"vlr_cst_unit" "vlr_cst_unit_ipi" "vlr_cst_unit_imp" "vlr_vnd_unit" "vlr_vnd_unit_ipi" "vlr_vnd_unit_imp" "vlr_lucro_unit"
"vlr_vnd_unit_b2b" "descricao" "area_user" "area_calc" "vlr_cst_nota_unit" "vlr_cst_entrada_unit" "valor_difal_unit"
"vlr_credito_icms_unit" "aliq_inter" "aliq_interna" "perc_difal" "vlr_frete_b2b_unit" "vlr_st_unit" "vlr_custo_fiscal_unit"
"eh_importado" "perc_margem_real" "Descricao" } } as $itemS } response = {totais: $recalc.totais, ORCA_1: $Orca_1, itemS: $itemS} }
```

Response: A função Orcamento_Recalcular_Totais faz o recálculo financeiro completo e dinâmico de um orçamento comercial (B2B/B2C), considerando impostos, frete, margem de lucro e descontos, atualizando tanto o cabeçalho quanto cada item no banco de dados.

O fluxo de execução ocorre em 6 etapas principais:

1. Consulta Inicial e Soma dos Custos Base

- Busca as informações do orçamento (Orca) e lista todos os seus itens associados.

- Calcula o custo de nota base de cada item (`vlr_cst_nota_unit * qtd`) e obtém o `total_custo_nota` acumulado do orçamento.
- Define a margem de markup alvo (usa `newMargem` se fornecida, ou a margem cadastrada no orçamento).

2. Cálculo do Frete B2B

- Executa a função `fCalculaFrete` passando o custo total de nota e um frete mínimo estipulado em R\$ 52,00.

3. Rateio, Impostos e Preço de Venda Através de um script de processamento:

- **Rateia o Frete B2B:** Proporcionalmente ao custo de nota de cada item em relação ao total.
- **Compos o Custo Fiscal:** `Custo Nota + DIFAL - Crédito ICMS + ST`.
- **Calcula o Custo de Entrada:** `Custo Fiscal + Frete B2B Rateado`.
- **Calcula a Venda Bruta e Desconto:** Aplica o markup sobre o custo de entrada para gerar a venda bruta. O desconto geral do orçamento é rateado proporcionalmente entre os itens.
- **Obtém a Venda Líquida e Margem Real:** Calcula o valor unitário final de venda (`vlr_vnd_unit`), o lucro por item e a margem real final (`perc_margem_real`).

4. Atualização dos Itens (item)

- Percorre cada item e atualiza no banco de dados os novos custos calculados, preço unitário de venda, lucro e frete B2B rateado.

5. Atualização do Cabeçalho (Orca)

- Grava no registro do orçamento os totais consolidados: custo total (`cst_tot`), venda total (`vnd_tot`), lucro total (`luc_tot`), vendas com frete B2C, acumulados de impostos (ST, DIFAL, Crédito ICMS) e a margem geral efetiva.

6. Retorno dos Dados

- Faz uma consulta final trazendo os itens atualizados com um `join` para incluir descrições formatadas (Material, Linha, Tipo, Nível, Borda) e retorna o JSON final contendo os totais, os dados do orçamento e a lista de itens.

User prompt: Quando tem um desconto ela preve algo?

Response: Sim, quando existe um desconto cadastrado no orçamento (`orca.desconto`), a função prevê **duas etapas principais** para tratá-lo:

1. Rateio Proporcional entre os Itens

Em vez de subtrair o desconto de forma bruta apenas no total final, a função distribui o desconto entre os itens **proporcionalmente ao valor de venda bruta** de cada um:

1. Calcula a Venda Bruta de cada item:

$$\text{Venda Bruta do Item} = \text{Custo de Entrada Total} \times (1 + \text{Markup})$$

2. Determina a proporção (ratio) do item no total:

$$\text{Ratio} = \frac{\text{Venda Bruta do Item}}{\text{Venda Bruta Total do Orçamento}}$$

3. Aplica o Desconto Rateado:

$$\text{Desconto do Item} = \text{Desconto Total} \times \text{Ratio}$$

4. Obtém a Venda Líquida e o Preço Unitário de Venda:

$$\text{Venda Líquida do Item} = \text{Venda Bruta do Item} - \text{Desconto do Item}$$

$$\text{vlr_vnd_unit} = \frac{\text{Venda Líquida do Item}}{\text{Qtd}}$$

2. Recálculo da Margem Real e do Markup Efetivo

Como o desconto reduz a receita bruta, ele afeta diretamente o lucro final. A função recalcula os indicadores financeiros considerando essa redução:

- **Lucro Unitário e Margem Real do Item:**

Calcula o lucro final após o desconto (`vlr_vnd_unit - vlr_cst_entrada_unit`) e ajusta a porcentagem da margem real (`perc_margem_real`).

- **Markup Efetivo Global (markup_efetivo / margem):**

Recalcula o markup real do orçamento consolidado considerando a venda total com desconto (`vnd_tot`):

$$\text{Markup Efetivo} = \frac{\text{vnd_tot} - \text{cst_tot}}{\text{cst_tot}} \times 100$$

Resumo do impacto nos campos atualizados:

- **Nos Itens (item):** O campo `vlr_vnd_unit` já é salvo no banco **com o desconto aplicado** (valor líquido unitário), o que também afeta diretamente `vlr_lucro_unit` e `perc_margem_real`.
- **No Cabeçalho (Orca):** O campo `vnd_tot` recebe o valor total de vendas já subtraído do desconto (`total_venda_bruta - desconto`), refletindo o novo lucro total (`luc_tot`) e a margem global reajustada.

User prompt: pode me dar um exemplo numerico?

Response: Para ilustrar como o desconto é rateado e recalcula a venda final, imagine um orçamento com **2 itens** e um **desconto total de R\$ 50,00**.

Cenário Exemplo

- Item A (Qty: 2): Custo de entrada total = R\$ 200,00
- Item B (Qty: 1): Custo de entrada total = R\$ 100,00
- Markup Alvo: 50%
- Desconto Total (orca.desconto): R\$ 50,00

Passo 1: Venda Bruta (sem desconto)

Aplica-se o markup sobre o custo de entrada de cada item:

- Item A: $R\$200,00 \times (1 + 0,50) = R\$300,00$
- Item B: $R\$100,00 \times (1 + 0,50) = R\$150,00$
- Venda Bruta Total: $R\$300,00 + R\$150,00 = R\$450,00$

Passo 2: Rateio do Desconto Proporcional

O código calcula o peso de cada item no total da venda bruta para distribuir os R\$ 50,00 de desconto:

- Item A:

$$\text{Proporção} = \frac{300}{450} = 66,67\%$$
$$\text{Desconto Rateado} = R\$50,00 \times 66,67\% = R\$33,33$$
- Item B:

$$\text{Proporção} = \frac{150}{450} = 33,33\%$$
$$\text{Desconto Rateado} = R\$50,00 \times 33,33\% = R\$16,67$$

Passo 3: Venda Líquida e Preço Unitário (vlr_vnd_unit)

Subtrai o desconto rateado e divide pela quantidade de cada item:

- Item A (2 unidades):

$$\text{Venda Líquida Item A} = R\$300,00 - R\$33,33 = R\$266,67$$
$$\text{vlr_vnd_unit} = \frac{266,67}{2} = \text{R\$133,335}$$
- Item B (1 unidade):

$$\text{Venda Líquida Item B} = R\$150,00 - R\$16,67 = R\$133,33$$
$$\text{vlr_vnd_unit} = \frac{133,33}{1} = \text{R\$133,33}$$

Passo 4: Recálculo dos Totais do Orçamento (Orca)

Métrica	Sem Desconto	Com Desconto	Impacto
Custo Total (cst_tot)	R\$ 300,00	R\$ 300,00	Mantido
Venda Total (vnd_tot)	R\$ 450,00	R\$ 400,00	- R\$ 50,00
Lucro Total (luc_tot)	R\$ 150,00	R\$ 100,00	- R\$ 50,00
Markup Efetivo (margem)	50,00%	33,33%	$\frac{400-300}{300} \times 100$
Margem Real (margem_real_total)	33,33%	25,00%	$\frac{100}{400} \times 100$

Resumo

O desconto de R\$ 50,00 reduziu a venda unitária de cada item e rebaixou o **markup efetivo do orçamento de 50% para 33,33%**, ajustando a lucratividade real automaticamente.

User prompt: E se eu tive um frete B2C ele faz alguma coisa?

Response: Sim. Se houver um valor de frete B2C informado no orçamento (orca.frtB2C), a função inclui esse valor apenas no totalizador final de venda do orçamento (vnd_B2B_B2C_tot), sem alterar os custos ou preços unitários dos itens.

Aqui está o detalhamento de como ele é tratado:

1. No Cabeçalho do Orçamento (Orca)

A função calcula o campo `vnd_B2B_B2C_tot` adicionando o frete B2C ao total líquido de venda:

$$vnd_B2B_B2C_tot = vnd_tot + frtB2C$$

- `vnd_tot`: Total das vendas dos itens (já com o desconto rateado retirado).
- `vnd_B2B_B2C_tot`: O valor total que o cliente final (B2C) pagará, incluindo a cobrança do frete de entrega.

2. O que o Frete B2C **NÃO** altera

Diferente do Frete B2B (que é diluído no custo de cada produto) ou do Desconto (que altera a venda unitária de cada item):

- **Não afeta o Preço Unitário** (`vlr_vnd_unit`): O valor cobrado por unidade do produto nos itens continua o mesmo.
- **Não afeta o Lucro Bruto dos Itens** (`vlr_lucro_unit`): Como o frete B2C é considerado um repasse/serviço de entrega ao cliente final, a margem de lucro calculada sobre os produtos não é alterada por ele.
- **Não afeta o Custo de Entrada dos Itens** (`vlr_cst_entrada_unit`): Apenas o frete B2B (frete de compra/fornecedor) entra na composição do custo do produto.

Exemplo Rápido

Usando os números do exemplo anterior:

- **Venda Total dos Itens** (`vnd_tot`): R\$ 400,00
- **Frete B2C** (`frtB2C`): R\$ 30,00

Resultado gerado no cabeçalho:

- `vnd_B2B_tot` = **R\$ 400,00** (*total da venda dos produtos*)
- `vnd_B2B_B2C_tot` = 400,00 + 30,00 = **R\$ 430,00** (*total final a cobrar do cliente*)

User prompt: E se for alterada a margem (markup) de 100 para 80 o que ocorre?

Response: Se a margem (markup) for alterada de **100% para 80%**, a função recalculará todo o valor de venda do orçamento no bloco Lambda.

A alteração do markup aciona uma reestruturação em **cadeia em 4 etapas**:

1. Aplicação da Nova Margem

A função identifica que `input.newMargem` (80) foi informado, ignorando a margem antiga do orçamento (100) e assumindo *markupAlvo* = 80.

2. Recálculo da Venda Bruta e do Desconto

- **Venda Bruta de cada item**: passa a ser calculada com 80% sobre o custo de entrada em vez de 100%:

$$Venda\ Bruta = Custo\ de\ Entrada \times (1 + 0,80)$$

- **Proporção do Desconto**: Como a venda bruta de todos os itens diminuiu, a proporção relativa entre eles é mantida, mas a *massa total de venda bruta* do orçamento cai.

3. Ajuste do Preço Unitário nos Itens (`item`)

Cada item do orçamento tem seus campos unitários recalculados no banco de dados:

- `vlr_vnd_unit` (**Preço de Venda Unitário**): Diminui, pois a margem aplicada sobre o custo caiu de 100% para 80%.
- `vlr_lucro_unit` (**Lucro Unitário**): Reduz diretamente, já que o custo de entrada permanece fixo, mas o preço de venda caiu.
- `perc_margem_real` (**Margem Real Unitária**): É reajustada para baixo.
(Exemplo: com markup de 100%, a margem real sobre a venda sem desconto é de 50%. Com markup de 80%, a margem real cai para 44,44%).

4. Atualização dos Totais no Cabeçalho (`Orca`)

Todos os acumulados de receita e margem do orçamento são reduzidos:

- `vnd_tot` (**Venda Total**): Cai (refletindo os novos preços unitários menores).
- `luc_tot` (**Lucro Total**): Diminui proporcionalmente.
- `vnd_B2B_B2C_tot`: Diminui (pois é a soma do novo `vnd_tot` menor + `frtB2C`).
- `margem` (**Markup Efetivo Global**): Atualizado para refletir o novo markup final (caso haja desconto, o markup efetivo ficará um pouco abaixo dos 80%).
- `margem_real_total`: É atualizada para a nova margem percentual calculada sobre a nova receita total.

Exemplo Prático Comparativo

Imagine um produto com **Custo de Entrada = R\$ 100,00** (Sem desconto no orçamento):

Métrica	Com Margem 100%	Com Margem 80%	Diferença
Custo de Entrada	R\$ 100,00	R\$ 100,00	<i>Inalterado</i>
Venda Unitária (<code>vlr_vnd_unit</code>)	R\$ 200,00	R\$ 180,00	- R\$ 20,00 (-10%)

Métrica	Com Margem 100%	Com Margem 80%	Diferença
Lucro Unitário (vlr_lucro_unit)	R\$ 100,00	R\$ 80,00	- R\$ 20,00 (-20%)
Margem Real (perc_margem_real)	50,00%	44,44%	- 5,56%

Resumo: Os custos (nota, fiscal e frete B2B) continuam exatamente os mesmos. O que se altera são **os preços finais de venda, os lucros e os indicadores percentuais de margem**, que são reduzidos tanto na tabela de itens quanto no cabeçalho do orçamento.

User prompt: aparentemente está tudo certo essa função. certo?

Response: A função está bem construída e tem uma estrutura lógica completa para regras de negócio complexas. No entanto, **ela possui alguns pontos de atenção e riscos de bugs que você deve avaliar antes de colocá-la em produção.**

1. Riscos de Bugs e Comportamentos Indesejados

- Ajuste Automático da Margem de Todos os Itens (markupAlvo)**
Se você passar o parâmetro `newMargem`, a função sobrescreve o markup de **todos** os itens com esse novo valor. Se o orçamento tinha itens com margens individuais diferentes (ex: item A com 50% e item B com 30%), ao passar `newMargem`, essa diferenciação individual **é perdida**.
- Persistência do Novo Markup no Cabeçalho**
Ao calcular `markup_efetivo` no Lambda, se houver um desconto, esse markup será menor que o `inputMargem`. Esse valor recalculado é gravado no banco no campo `Orca.margem`. Se você rodar a função novamente no futuro **sem** o parâmetro `newMargem`, a lógica usará o `Orca.margem` (que já foi rebaixado pelo desconto anterior) como nova base de markup, gerando um efeito em cascata que **vai reduzindo o preço unitário a cada recálculo**.

2. Detalhes de Implementação e Código

- Uso Inconsistente de Variáveis no Lambda 2**

```
Linha 82: const st_unit = Number(item.vlr_st_unit) || Number(item.vlr_st) || 0;
```

O campo `item.vlr_st` **não existe** na lista de campos retornados na consulta inicial da tabela `item` (o SQL de busca pede `vlr_st_unit`). Embora o fallback garanta que o código não quebre, o campo secundário é redundante.
- Formatos das Propriedades no db.edit**
Algumas atualizações gravam valores formatados com `.toFixed(4)`. Como o método `.toFixed()` do JavaScript devolve uma **string** (ex: "133.3333" em vez de número 133.3333), certifique-se de que a camada de banco de dados (como o Xano ou similar) converte automaticamente essa string para um tipo numérico (`decimal/float`) sem gerar erro de tipo.

3. Falta de Tratamento para Casos Extremos (Edge Cases)

- Orçamentos com Custo Total Zero**
Se todos os itens tiverem custo de nota igual a R\$ 0,00, a variável `total_custo_nota` será 0. No rateio do Frete B2B, existe a trava `total_custo_nota > 0 ? ... : 0`, o que evita a divisão por zero. O frete B2B ficará acumulado no totalizador, mas o valor unitário rateado será R\$ 0,00.
- Tratamento do Desconto Maior que a Venda Bruta**
Se o valor de `orca.desconto` for superior ao valor total de venda bruta, o valor de `vnd_tot` e `vlr_vnd_unit` ficarão negativos, gerando margens reais e lucros negativos. Se o seu sistema não permite orçamentos no prejuízo ou com valor negativo, vale adicionar uma validação antes do cálculo.

Resumo

O **fluxo financeiro e a matemática estão corretos**. O ponto crítico principal a revisar é o **comportamento do desconto acumulado sobre o campo `Orca.margem`**, garantindo que recálculos sucessivos não rebaixem o markup repetidamente.

User prompt: o que vc pensou em fazer?

Response: O problema central está nesta linha do segundo bloco Lambda:

```
const markup_efetivo = cst_tot > 0 ? ((vnd_tot - cst_tot) / cst_tot) * 100 : markupAlvo;
```

E no fato de que o valor de `markup_efetivo` é gravado no banco tanto nos itens (`upd.margem`) quanto no cabeçalho (`Orca.margem`).

O Bug do "Efeito Cascata"

Imagine a seguinte sequência de execuções:

- Primeira Execução (sem desconto):**
 - Custo Total = R\$ 100,00
 - Markup Alvo (`Orca.margem`) = **50%**
 - Venda = R\$ 150,00
 - Salva `Orca.margem = 50%`
- Segunda Execução (adicionando Desconto de R\$ 10,00):**
 - Venda Líquida = R\$ 140,00 (150 - 10)
 - A função calcula o `markup_efetivo`: $\frac{140-100}{100} \times 100 = \mathbf{40\%}$

- Salva `Orca.margem = 40%` no banco de dados.

3. Terceira Execução (qualquer atualização simples, ex: adicionar/alterar um item):

- A função lê `Orca.margem` do banco, que agora é **40%** (em vez dos 50% originais).
- Ela calcula a nova venda bruta baseada em 40%: R\$ 140,00.
- Aplica o desconto de R\$ 10,00 novamente sobre os R\$ 140,00.
- Nova Venda Líquida = R\$ 130,00!
- O preço **caiu novamente sem que ninguém tenha alterado a margem**.

A Solução Proposta

Para corrigir isso sem mudar a lógica do seu sistema, devemos **separar o Markup Base do Markup Efetivo**:

1. Manter o `Orca.margem` Intacto (Markup Base):

Quando houver desconto, o campo `Orca.margem` no banco **não** deve ser sobrescrito pelo markup rebaixado. Ele deve guardar sempre o markup comercial original que a empresa definiu (ex: 50%).

2. Gravar a Margem Efetiva em um Campo Separado (opcional):

Se você precisa exibir a margem após o desconto na tela, o ideal é salvar em um campo próprio (ex: `margem_efetiva` ou usar o `perc_margem_real`), mantendo `Orca.margem` apenas com o valor base.

3. Garantir a Precedência do `newMargem`:

Se o usuário enviar `newMargem` via `$input`, aí sim atualizamos a margem base do orçamento.

Como Faria a Alteração no Código

No **bloco 3 (Lambda)**, o retorno do cálculo do markup seria ajustado assim:

```
// O markup base para os próximos cálculos continua sendo o markupAlvo original
const markup_base_para_salvar = (inputMargem > 0) ? inputMargem : (Number(orca.margem) || 0);

return {
  itensParaUpdate: itensComVendaFinal.map(i => ({
    id                : i.id,
    // ...
    // Grava no item o markup base (ou o efetivo, dependendo da sua regra visual)
    margem            : Number(markup_base_para_salvar.toFixed(4)),
    perc_margem_real   : Number(i.perc_margem_real.toFixed(4))
  })),
  totais: {
    // ...
    // Grava no cabeçalho o Markup BASE, garantindo a idempotência dos cálculos futuros
    margem            : Number(markup_base_para_salvar.toFixed(4)),
    markup_efetivo     : Number(markup_efetivo.toFixed(4)),
    margem_real_total  : Number(margem_real_total.toFixed(4)),
    // ...
  }
};
```

Dessa forma, o orçamento pode ser recalculado **dezenas de vezes** com desconto e frete que o valor de venda **permanecerá sempre estável**.

User prompt: Não pode... o desconto se for dado outro deve zerar o anterior primeiro e depois dar o novo desconto

Response: Entendido. Para garantir que o cálculo seja **idempotente** (ou seja, executar a função várias vezes não acumule descontos ou reduza a margem infinitamente), a regra precisa tratar o desconto como um valor fixo que substitui o anterior, preservando sempre a **Margem Base Original** do orçamento.

Como ajustar a lógica para funcionar corretamente:

1. Separar a Margem Base do Markup Efetivo:

- A margem usada para precificar o custo de entrada (ex: 50%) deve ser mantida como a referência do orçamento.
- O desconto entra no final calculando uma **venda líquida**, gerando o *markup efetivo*, mas **sem sobrescrever** a margem base que deu origem aos preços.

2. Substituição do Desconto:

- Quando um novo desconto é informado ou o orçamento é recalculado, a função sempre reaplica a margem base sobre os custos atuais para recalcular a **Venda Bruta**, e só então aplica o valor atual de `orca.desconto` (que já é o desconto atualizado do cabeçalho).

Alteração no Bloco Lambda (Bloco 3)

No código do seu Lambda, a alteração principal consiste em salvar no campo `margem` do cabeçalho e dos itens o **markupAlvo (Margem Base)**, em vez do `markup_efetivo`.

```
// --- DENTRO DO BLOCO 3 (api.lambda) ---

// 1. Identifica a Margem Base Comercial
const inputMargem = $input ? Number($input.newMargem) : 0;
const markupAlvo = (inputMargem > 0) ? inputMargem : (Number(orca.margem) || 0);

// ... (Cálculo dos Custos de Entrada, Frete B2B e Impostos) ...
```

```
// 2. Calcula a Venda Bruta SEMPRE a partir do Markup Base
const itensComVendaBruta = itensComCustoEntrada.map(item => {
  const venda_bruta_item = item.custo_entrada_total * (1 + (markupAlvo / 100));
  return { ...item, venda_bruta_item };
});

const total_venda_bruta = itensComVendaBruta.reduce((acc, i) => acc + i.venda_bruta_item, 0);

// 3. Aplica o Desconto do Cabeçalho (que substitui qualquer desconto anterior)
const itensComVendaFinal = itensComVendaBruta.map(item => {
  const ratioVenda = total_venda_bruta > 0 ? (item.venda_bruta_item / total_venda_bruta) : 0;
  const desconto_rateado_item = desconto * ratioVenda;
  const venda_liquida_item = item.venda_bruta_item - desconto_rateado_item;

  const vlr_vnd_unit = venda_liquida_item / item.qtd;
  const vlr_lucro_unit = vlr_vnd_unit - item.vlr_cst_entrada_unit;
  const perc_margem_real = vlr_vnd_unit > 0 ? (vlr_lucro_unit / vlr_vnd_unit) * 100 : 0;

  return {
    ...item,
    venda_liquida_item,
    vlr_vnd_unit,
    vlr_lucro_unit,
    perc_margem_real
  };
});

const vnd_tot = total_venda_bruta - desconto;
const luc_tot = vnd_tot - cst_tot;
const markup_efetivo = cst_tot > 0 ? ((vnd_tot - cst_tot) / cst_tot) * 100 : markupAlvo;
const margem_real_total = vnd_tot > 0 ? (luc_tot / vnd_tot) * 100 : 0;

return {
  itensParaUpdate: itensComVendaFinal.map(i => ({
    id : i.id,
    vlr_cst_nota_unit : Number(i.cst_nota_unit.toFixed(4)),
    vlr_st_unit : Number(i.st_unit.toFixed(4)),
    vlr_custo_fiscal_unit : Number(i.vlr_custo_fiscal_unit.toFixed(4)),
    vlr_frete_b2b_unit : Number(i.vlr_frete_b2b_unit.toFixed(4)),
    vlr_cst_entrada_unit : Number(i.vlr_cst_entrada_unit.toFixed(4)),
    vlr_vnd_unit : Number(i.vlr_vnd_unit.toFixed(4)),
    vlr_vnd_unit_b2b : Number(i.vlr_vnd_unit.toFixed(4)),
    vlr_lucro_unit : Number(i.vlr_lucro_unit.toFixed(4)),
    // SALVA A MARGEM BASE (markupAlvo) NO ITEM, NÃO A REBAIXADA
    margem : Number(markupAlvo.toFixed(4)),
    perc_margem_real : Number(i.perc_margem_real.toFixed(4))
  })),
  totais: {
    cst_tot : Number(cst_tot.toFixed(4)),
    vlr_st_tot : Number(vlr_st_tot.toFixed(4)),
    valor_difal_tot : Number(valor_difal_tot.toFixed(4)),
    vlr_credito_icms_tot : Number(vlr_credito_icms_tot.toFixed(4)),
    vlr_custo_fiscal_tot : Number(vlr_custo_fiscal_tot.toFixed(4)),
    vnd_tot : Number(vnd_tot.toFixed(4)),
    luc_tot : Number(luc_tot.toFixed(4)),
    vnd_B2B_tot : Number(vnd_tot.toFixed(4)),
    vnd_B2B_B2C_tot : Number((vnd_tot + frtB2C).toFixed(4)),
    // SALVA O MARKUP BASE NO CABEÇALHO PARA PERMITIR REPETIÇÃO DO CÁLCULO
    margem : Number(markupAlvo.toFixed(4)),
    markup_efetivo : Number(markup_efetivo.toFixed(4)),
    margem_real_total : Number(margem_real_total.toFixed(4)),
    frete_b2b_total : Number(frete_b2b_total.toFixed(4)),
    desconto : Number(desconto.toFixed(4)),
    frtB2C : Number(frtB2C.toFixed(4)),
    total_itens : itensComVendaFinal.length
  }
});
```

O que isso resolve na prática?

- **Zeramento/Substituição de Desconto:** Se você alterar `Orca.desconto` de R\$ 50 para R\$ 20 (ou R\$ 0) na tabela `Orca` e rodar a função, a **Venda Bruta** será sempre recalculada do zero usando os 50% de markup inicial. O desconto anterior de R\$ 50 deixa de existir e o novo de R\$ 20 é aplicado sobre o total bruto limpo.
- **Estabilidade Financeira:** Chamar o recálculo 10 vezes seguidas produzirá exatamente o mesmo resultado numérico todas as vezes.

User prompt: vc pode me dar um exemplo que vc falou acima?

Response: Para entender como a correção impede o efeito em cascata, acompanhe o comparativo do que acontece quando o orçamento passa por **3 atualizações seguidas** com um **desconto de R\$ 50,00**.

Mapeamento dos Dados Iniciais

- **Custo de Entrada do Orçamento (cst_tot):** R\$ 100,00
- **Markup Comercial Desejado:** 50%

- Novo Desconto no Orçamento (`orca.desconto`): R\$ 50,00

Cenário 1: Com o Código Antigo (Com Bug do Efeito Cascata)

Na lógica antiga, a função salvava o **markup rebaixado pelo desconto** de volta no banco de dados (`Orca.margem`).

1. Rodada 1 (Primeiro Recálculo):

- Venda Bruta (base 50%): $100 \times 1,50 = \text{R\$ } 150,00$
- Venda Líquida com Desconto: $150 - 50 = \text{R\$ } 100,00$
- O código calcula o Markup Efetivo: $\frac{100-100}{100} = 0\%$
- **Erro gravado no banco:** Salva `Orca.margem = 0%`.

2. Rodada 2 (Qualquer alteração simples no orçamento, ex: mudou o CEP ou atualizou um item):

- O código lê a margem do banco (`Orca.margem`), que agora é **0%**!
- Nova Venda Bruta (base 0%): $100 \times 1,00 = \text{R\$ } 100,00$
- Aplica o mesmo desconto de R\$ 50,00 novamente: $100 - 50 = \text{R\$ } 50,00$
- **Resultado:** A venda caiu para R\$ 50,00 e a loja passou a ter **prejuízo**, mesmo sem ninguém mexer nos preços!

Cenário 2: Com o Código Corrigido (Preservando a Margem Base)

Na lógica ajustada, o campo `Orca.margem` é mantido fixo em **50%** e o desconto de R\$ 50,00 sempre entra abatendo a venda bruta original.

1. Rodada 1 (Primeiro Recálculo):

- Venda Bruta (base 50%): $100 \times 1,50 = \text{R\$ } 150,00$
- Venda Líquida com Desconto: $150 - 50 = \text{R\$ } 100,00$
- **Salva no banco:** `Orca.margem = 50%` (preservada) e `markup_efetivo = 0%`.

2. Rodada 2 (Alteração simples ou atualização do orçamento):

- O código lê a margem do banco, que continua sendo **50%**.
- Recalcula a Venda Bruta limpa: $100 \times 1,50 = \text{R\$ } 150,00$
- Re-aplica o desconto de R\$ 50,00: $150 - 50 = \text{R\$ } 100,00$
- **Resultado:** A venda permanece **estável em R\$ 100,00**.

3. Rodada 3 (Mudou o desconto para R\$ 20,00 no banco):

- O código lê a margem base de **50%**.
- Recalcula a Venda Bruta limpa: $100 \times 1,50 = \text{R\$ } 150,00$
- O desconto antigo de R\$ 50 é totalmente ignorado e entra o novo de R\$ 20: $150 - 20 = \text{R\$ } 130,00$
- **Resultado:** O desconto anterior foi **zerado e substituído perfeitamente**.

Tabela Comparativa de Resultados

Etapas / Ação	Código Antigo (Com Bug)	Código Corrigido
Venda Inicial (Sem desconto)	R\$ 150,00	R\$ 150,00
1º Recálculo (Desconto R\$ 50)	R\$ 100,00	R\$ 100,00
2º Recálculo (Sem alterar nada)	R\$ 50,00 (Erro/Prejuízo)	R\$ 100,00 (Estável)
3º Recálculo (Alterou desc. p/ R\$ 20)	R\$ 30,00 (Erro de cálculo)	R\$ 130,00 (Substituição perfeita)

User prompt: certo eu pensei em um sistema dinamico aonde na tela ele tem uns campos para mudar o valor final de venda = ex 4586,02 ele quer arredondar para 4500 dá então o sistema pega esse valor altera para 4500 e retorna uma nova margem sobre o valor de custo essa dinamica tb poderia ocorrer alterando o lucro para cima e para baixo -> encontra a nova margem e ou no sistema ele pode alterar a Margem Daria para fazer?

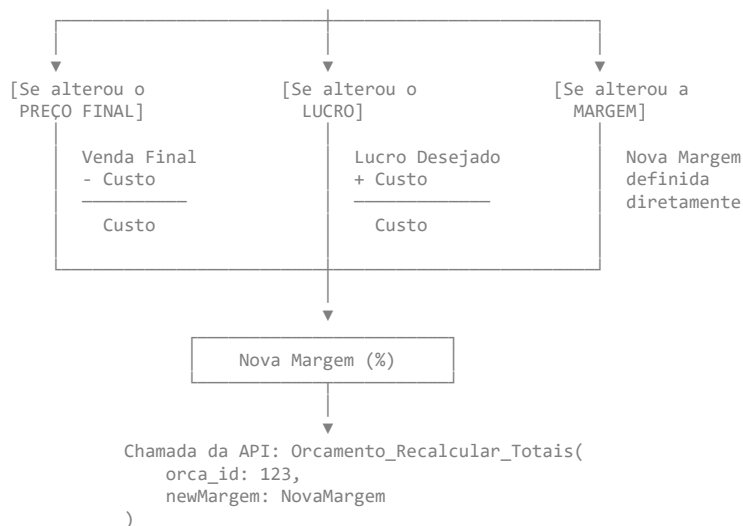
Response: Sim, é perfeitamente possível e é uma excelente funcionalidade para negociação comercial. Para funcionar bem sem quebrar o banco de dados, o segredo é **tratar todos esses campos como "gatilhos de entrada" que resultam na descoberta da nova Margem (Markup Base)** do orçamento.

Na prática, todas essas ações do usuário (mudar Preço Final, mudar Lucro ou mudar Margem diretamente) são apenas caminhos diferentes para encontrar a mesma variável: a **nova newMargem**.

Como funciona a matemática no Front-end / Back-end

Você precisa apenas de **três fórmulas matemáticas inversas** para calcular a margem antes de chamar o recálculo do orçamento:

Custo Total (`cst_tot`)



Mapeamento das Três Ações

1. Alterando pelo Preço Final de Venda

Se o orçamento está em **R\$ 4.586,02** com um custo total de **R\$ 3.000,00** e o vendedor digita **R\$ 4.500,00**:

$$\text{Nova Margem (\%)} = \left(\frac{\text{Venda Desejada} - \text{Custo Total}}{\text{Custo Total}} \right) \times 100$$

$$\text{Nova Margem (\%)} = \left(\frac{4.500 - 3.000}{3.000} \right) \times 100 = 50,00\%$$

2. Alterando pelo Lucro

Se o lucro atual é R\$ 1.586,02 e o vendedor quer definir um lucro fixo de **R\$ 1.200,00**:

$$\text{Nova Venda} = \text{Custo Total} + \text{Lucro Desejado} = 3.000 + 1.200 = 4.200$$

$$\text{Nova Margem (\%)} = \left(\frac{1.200}{3.000} \right) \times 100 = 40,00\%$$

3. Alterando pela Margem Direta

O vendedor simplesmente digita que quer trabalhar com **45%** de markup.

Como Estruturar na Sua Aplicação

Para que fique fluido e profissional para o usuário:

1. Na Tela (Front-end):

Você coloca 3 campos editáveis: [Preço Final], [Lucro] e [Margem %].

2. Ao mudar qualquer um dos 3:

O sistema descobre a `newMargem` equivalente usando a fórmula do custo total e envia para a sua função backend `Orcamento_Recalcular_Totais(orca_id, newMargem)`.

3. No Back-end:

Sua função recebe a `newMargem`, recalcula os impostos, rateia o frete B2B e atualiza **todos os itens proporcionalmente**, retornando a nova tabela de preços e os totais perfeitos.

Isso evita ter que criar 3 funções diferentes no banco de dados. Uma única função de recálculo (a `Orcamento_Recalcular_Totais`) atende todos os cenários.

User prompt: Pode dar um exemplo de código JavaScript de como implementar essas conversões de Preço, Lucro e Margem no Front-end?

Response: Aqui está um exemplo completo em JavaScript de como capturar a alteração em qualquer um dos três campos (**Preço Final**, **Lucro** ou **Margem %**), calcular os valores equivalentes e disparar o recálculo para a sua API:

```
// Estado atual do orçamento recebido da API
let orcamento = {
  orca_id: 1024,
  cst_tot: 3000.00, // Custo Total (Custo de Entrada + Frete B2B)
  vnd_tot: 4586.02, // Venda Total Atual
  luc_tot: 1586.02, // Lucro Total Atual
  margem: 52.8673 // Margem/Markup Atual (%)
};

/**
 * Função principal que recalcula os 3 pilares a partir do campo alterado
 * @param {string} campoAlterado - 'preco' | 'lucro' | 'margem'
 * @param {number} valorDigitado - Novo valor inserido pelo usuário
 */
```

```

*/
function handleMudancaFinanceira(campoAlterado, valorDigitado) {
  const custoTotal = Number(orcamento.cst_tot) || 0;

  if (custoTotal <= 0) {
    console.error("Custo total zerado. Não é possível calcular a margem.");
    return;
  }

  let novaMargem = 0;
  let novaVenda = 0;
  let novoLucro = 0;

  switch (campoAlterado) {
    case 'preco':
      // Ex: Usuário digitou 4500.00
      novaVenda = Number(valorDigitado);
      novoLucro = novaVenda - custoTotal;
      novaMargem = ((novaVenda - custoTotal) / custoTotal) * 100;
      break;

    case 'lucro':
      // Ex: Usuário digitou 1200.00
      novoLucro = Number(valorDigitado);
      novaVenda = custoTotal + novoLucro;
      novaMargem = (novoLucro / custoTotal) * 100;
      break;

    case 'margem':
      // Ex: Usuário digitou 50 (%)
      novaMargem = Number(valorDigitado);
      novaVenda = custoTotal * (1 + (novaMargem / 100));
      novoLucro = novaVenda - custoTotal;
      break;

    default:
      return;
  }

  // 1. Atualização visual instantânea nos inputs (Feedback rápido para o usuário)
  atualizarCamposNaTela({
    preco: novaVenda.toFixed(2),
    lucro: novoLucro.toFixed(2),
    margem: novaMargem.toFixed(2)
  });

  // 2. Envio da nova margem para a API de recálculo
  enviarParaBackend(orcamento.orca_id, Number(novaMargem.toFixed(4)));
}

/**
 * Atualiza os valores visuais no HTML
 */
function atualizarCamposNaTela(valores) {
  document.getElementById('inputPrecoFinal').value = valores.preco;
  document.getElementById('inputLucro').value = valores.lucro;
  document.getElementById('inputMargem').value = valores.margem;
}

/**
 * Chama a função no backend passando o orca_id e a nova margem encontrada
 */
async function enviarParaBackend(orca_id, newMargem) {
  try {
    const response = await fetch('/api/Orcamento_Recalcular_Totais', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        orca_id: orca_id,
        newMargem: newMargem
      })
    });
  });

  const data = await response.json();

  // Atualiza o estado local do orçamento com a resposta oficial do backend
  orcamento = data.totais;
  console.log("Orçamento recalculado com sucesso no servidor:", data);
} catch (error) {
  console.error("Erro ao enviar recálculo:", error);
}
}

```

Exemplo de Integração nos Inputs HTML

Você pode associar a função ao evento `onchange` ou ao perder o foco (`onblur`) de cada input:

```

<!-- Input de Venda Final -->
<label>Preço Final (R$):</label>

```

```

<input
  type="number"
  id="inputPrecoFinal"
  value="4586.02"
  step="0.01"
  onchange="handleMudancaFinanceira('preco', this.value)"
/>

<!-- Input de Lucro -->
<label>Lucro Total (R$):</label>
<input
  type="number"
  id="inputLucro"
  value="1586.02"
  step="0.01"
  onchange="handleMudancaFinanceira('lucro', this.value)"
/>

<!-- Input de Margem % -->
<label>Markup / Margem (%):</label>
<input
  type="number"
  id="inputMargem"
  value="52.87"
  step="0.01"
  onchange="handleMudancaFinanceira('margem', this.value)"
/>

```

Dica de Experiência do Usuário (UX)

Para evitar chamadas excessivas à API caso o usuário esteja digitando números no teclado, é recomendável usar o evento `onblur` (quando ele clica fora do campo) ou aplicar uma função **debounce** (que aguarda uns 500ms após o término da digitação) antes de disparar o `enviarParaBackend`.

User prompt: handleMudancaFinanceira neste caso o usuario iria brincar com os numeros se quisesse fechar ai enviara para o servidor?

Response: Sim, exatamente! Essa é a **melhor abordagem de UX (Experiência do Usuário)** para esse tipo de tela de negociação.

Em vez de disparar uma requisição para a API a cada número digitado ou campo alterado (o que deixaria a tela travando e sobrecarregaria o servidor), o usuário **"brinca" com os números localmente** no navegador. O envio para o servidor só acontece quando ele decide aplicar/confirmar.

Como estruturar esse fluxo em 2 etapas:

- Modo Simulação (100% Local / Front-end):** Conforme o usuário digita no Preço Final, Lucro ou Margem %, o JavaScript faz as contas instantaneamente na memória e atualiza a tela em milissegundos.
- Modo Aplicação (Envio ao Back-end):** Um botão como **"Aplicar Alterações"** ou **"Recalcular Orçamento"** pega a `novaMargem` calculada pela simulação e dispara a requisição para a função `Orçamento_Recalcular_Totais` no banco de dados.

Exemplo do Código Ajustado para Simulação + Confirmação

```

// Estado do orçamento vindo do banco
let orcamento = {
  orca_id: 1024,
  cst_tot: 3000.00,
  vnd_tot: 4586.02,
  luc_tot: 1586.02,
  margem: 52.8673
};

// Variável para guardar a margem simulada atual na memória
let margemSimulada = orcamento.margem;

/**
 * 1. Apenas SIMULA e atualiza a tela (SEM chamar API)
 */
function simularMudancaFinanceira(campoAlterado, valorDigitado) {
  const custoTotal = Number(orcamento.cst_tot) || 0;
  if (custoTotal <= 0) return;

  let novaVenda = 0;
  let novoLucro = 0;

  switch (campoAlterado) {
    case 'preco':
      novaVenda = Number(valorDigitado);
      novoLucro = novaVenda - custoTotal;
      margemSimulada = ((novaVenda - custoTotal) / custoTotal) * 100;
      break;

    case 'lucro':
      novoLucro = Number(valorDigitado);
      novaVenda = custoTotal + novoLucro;
      margemSimulada = (novoLucro / custoTotal) * 100;
      break;

    case 'margem':
      margemSimulada = Number(valorDigitado);

```

```

    novaVenda = custoTotal * (1 + (margemSimulada / 100));
    novoLucro = novaVenda - custoTotal;
    break;
}

// Atualiza apenas os campos na tela em tempo real
document.getElementById('inputPrecoFinal').value = novaVenda.toFixed(2);
document.getElementById('inputLucro').value = novoLucro.toFixed(2);
document.getElementById('inputMargem').value = margemSimulada.toFixed(2);
}

/**
 * 2. Envia para o servidor APENAS quando o usuário clica no botão
 */
async function aplicarAlteracaoNoServidor() {
  try {
    // Exibe um indicador de carregamento
    document.getElementById('btnAplicar').disabled = true;

    const response = await fetch('/api/Orcamento_Recalcular_Totais', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        orca_id: orcamento.orca_id,
        newMargem: Number(margemSimulada.toFixed(4)) // Envia a margem final definida na simulação
      })
    });

    const data = await response.json();

    // Atualiza os dados oficiais com a resposta do backend
    orcamento = data.totais;
    alert("Orçamento recalculado e salvo com sucesso!");

  } catch (error) {
    console.error("Erro ao salvar recálculo:", error);
    alert("Houve um erro ao atualizar o orçamento.");
  } finally {
    document.getElementById('btnAplicar').disabled = false;
  }
}

```

Exemplo da Interface (HTML)

Use o evento `oninput` nos campos para que a simulação aconteça **enquanto o usuário digita**:

```

<div class="painel-negociacao">
  <!-- Inputs de Simulação -->
  <label>Preço Final (R$):</label>
  <input type="number" id="inputPrecoFinal" value="4586.02" step="0.01" oninput="simularMudancaFinanceira('preco', this.value)" />

  <label>Lucro Total (R$):</label>
  <input type="number" id="inputLucro" value="1586.02" step="0.01" oninput="simularMudancaFinanceira('lucro', this.value)" />

  <label>Margem (%):</label>
  <input type="number" id="inputMargem" value="52.87" step="0.01" oninput="simularMudancaFinanceira('margem', this.value)" />

  <!-- Botão de Ação para Enviar ao Servidor -->
  <button id="btnAplicar" onclick="aplicarAlteracaoNoServidor()">
    Aplicar e Salvar no Orçamento
  </button>
</div>

```

Vantagens dessa abordagem:

- **Fluidez Total:** O vendedor pode testar dezenas de arredondamentos e valores de cabeça sem nenhum *delay* de rede.
- **Economia de Servidor:** Em vez de fazer 15 chamadas à API enquanto ele digita "4", "45", "450", "4500", você só faz **1 chamada** no final.
- **Segurança:** Evita gravar dados intermediários e parciais no banco de dados por engano.

User prompt: no caso nessa negociacao os itens ficariam staticos e bricariamos so com a parte do cabeçalho? ou alterariamos os itens tb?

Response: Sim, **os itens também devem ser atualizados!**

O cabeçalho do orçamento é apenas a **soma consolidada** do que está nos itens. Se você alterar o Preço Final de Venda no cabeçalho (de R\$ 4.586,02 para R\$ 4.500,00), os valores unitários de cada item **precisam mudar proporcionalmente**. Caso contrário, haverá uma divergência clássica no sistema: a soma da tabela de itens não baterá com o valor total mostrado no cabeçalho.

Como funciona essa dinâmica em cada etapa:

1. Na Tela (Front-end - Durante a Simulação)

Enquanto o vendedor está apenas "brincando" com os números:

- **Só a parte visual precisa mudar instantaneamente.** O JavaScript pode atualizar os totais do cabeçalho em tempo real e, se desejar, fazer um rateio visual prévio na tabela de itens da tela.

- Nada é enviado ao banco de dados ainda.

2. No Servidor (Back-end - Ao Clicar em "Aplicar e Salvar")

Quando o usuário clica para confirmar, a sua função `Orcamento_Recalcular_Totais` entra em ação e altera a base de dados de **duas formas**:

1. **Atualiza Item por Item** (`db.edit item`):
 - Aplica a nova margem (ex: 50%) sobre o custo de cada produto.
 - Recalcula o novo preço unitário de venda (`vlr_vnd_unit`).
 - Recalcula o novo lucro unitário (`vlr_lucro_unit`).
 - Ajusta a margem real individual (`perc_margem_real`).
2. **Atualiza o Cabeçalho** (`db.add_or_edit Orca`):
 - Grava os novos totais consolidados (`vnd_tot`, `luc_tot`, `margem`).

Por que isso é indispensável para o seu negócio?

- **Geração de Proposta/PDF:** Se o cliente pedir o orçamento impresso ou em PDF com os itens detalhados, os preços unitários já estarão ajustados para que a soma exata dê os R\$ 4.500,00 arredondados.
- **Faturamento/Emissão de Nota Fiscal:** Quando o orçamento for convertido em pedido/NF, a nota fiscal exige o valor unitário de cada item (`vlr_vnd_unit`). Se só o cabeçalho mudasse, a emissão da nota fiscal falharia ou seria emitida com valores incorretos.
- **Comissão e Lucratividade por Produto:** O relatório de margem por produto continuará preciso, refletindo o desconto/arredondamento concedido na negociação.

User prompt: Sim. Eu sei mas da forma como estamos pensando. No momento em que o vendedor está mechendo com os numero antes de submeter para o servidor

Response: Durante a fase de simulação no Front-end (antes de clicar em "Aplicar"), você pode escolher **duas abordagens** para a tabela de itens na tela:

Opção 1: Atualizar os Itens em Tempo Real na Tela (Recomendado)

Enquanto o vendedor altera o preço final no cabeçalho (ex: de R\$ 4.586,02 para R\$ 4.500,00), o JavaScript atualiza em tempo real não só o cabeçalho, mas **também a coluna de "Preço Unitário" e "Subtotal" de cada item** diretamente na tabela visual.

Como o JavaScript faz isso na tela (sem API):

1. Calcula a variação proporcional:

$$\text{Fator de Ajuste} = \frac{\text{Novo Total de Venda}}{\text{Venda Atual Total}} = \frac{4.500,00}{4.586,02} \approx 0,98124$$

2. Multiplica o preço de cada item na memória pelo fator:

$$\text{Novo Preço do Item} = \text{Preço Atual do Item} \times 0,98124$$

3. Atualiza os textos da tabela no HTML em milissegundos.

Vantagem: Transparência total. O vendedor já enxerga na hora quanto cada produto vai custar individualmente para fechar naquele valor arredondado.

Opção 2: Atualizar Apenas os Totais do Cabeçalho na Simulação

Os itens na tela continuam **estáticos com os preços originais** enquanto o vendedor digita no cabeçalho.

Apenas os cards/inputs do topo (`Preço Final`, `Lucro` e `Margem %`) mudam conforme ele simula. A tabela de itens só tem seus valores atualizados na tela **depois** que ele clica em "Aplicar" e a resposta da API retorna do servidor.

Vantagem: Implementação no Front-end um pouco mais simples e leve, pois não precisa manipular múltiplos nós do HTML durante a digitação.

Qual escolher?

Aspecto	Opção 1 (Itens Dinâmicos na Tela)	Opção 2 (Itens Estáticos na Tela)
UX / Experiência	Excelente (vendedor vê o impacto exato por item)	Boa (foco apenas no valor global da negociação)
Complexidade JS	Média (exige recalculas o array de itens no JS)	Baixa (manipula apenas 3 inputs de texto)
Resultado Final no Banco	Idêntico (o back-end atualizará tudo ao salvar)	Idêntico (o back-end atualizará tudo ao salvar)

Recomendação: A **Opção 1** costuma passar muito mais segurança para o vendedor, pois ele confirma o valor sabendo exatamente como cada produto ficou cotado antes de salvar.